FLUTTER LAB 3 REPORT

Hard-Coded Multi-Screen Travel App UI

Travel Explorer

Name: NIYONKURU Jean De La Croix ID: 223003235

Name: TUMUKUNDE Gentille ID: 223021200

Course: Mobile application systems and design

Year of study: Year 3

Department: Computer and Software Engineering

TABLE OF CONTENTS

1. Introduction

This report documents the development of Travel Explorer, a multi-screen Travel App built using Flutter. The app is entirely UI-focused with all data hard-coded in Dart files. No APIs, databases, or external data sources are used.

The app demonstrates advanced UI design techniques including complex layouts, widget composition, gradient overlays, box shadows, consistent theming, and navigation between three main screens with data passing.

Technologies: Flutter SDK, Dart programming language

Platform: Android / iOS / Web

Architecture: Organized by feature (data, models, screens, widgets)

2. Project Structure

The project follows a clean folder structure with separation of concerns. Each folder has a specific purpose:

File / Folder	Purpose
lib/main.dart	Entry point; applies theme and launches HomeScreen
lib/data/travel_data.dart	All hard-coded destinations, categories, prices, ratings
lib/data/app_theme.dart	Colors, gradients, shadows, text styles, ThemeData
lib/models/destination.dart	Destination data model class
lib/screens/home_screen.dart	Screen 1: Dashboard with search, categories, grid
lib/screens/detail_screen.dart	Screen 2: Destination details, rating, Book Now
lib/screens/booking_screen.dart	Screen 3: Booking form, summary, confirmation
lib/widgets/ (5 files)	Custom reusable widgets: RatingStars, DestinationCard, SectionTitle, C
assets/images/	6 destination images (Santorini, Kyoto, Swiss Alps, Bali, Paris, Machu P
pubspec.yaml	Flutter project configuration with asset declarations

3. Hard-Coded Data File

All data is stored in lib/data/travel_data.dart as required by the lab specification. The TravelData class contains:

- A list of 8 travel categories (Beaches, Mountains, Cities, Historic, Tropical, Winter, Adventure, Culinary) with emoji icons
- A list of 6 Destination objects, each containing: id, name, country, description, imageAsset, rating, price, category, isFavorite, and highlights
- Helper methods: featuredDestinations (filters favorites) and getByCategory (filters by category)

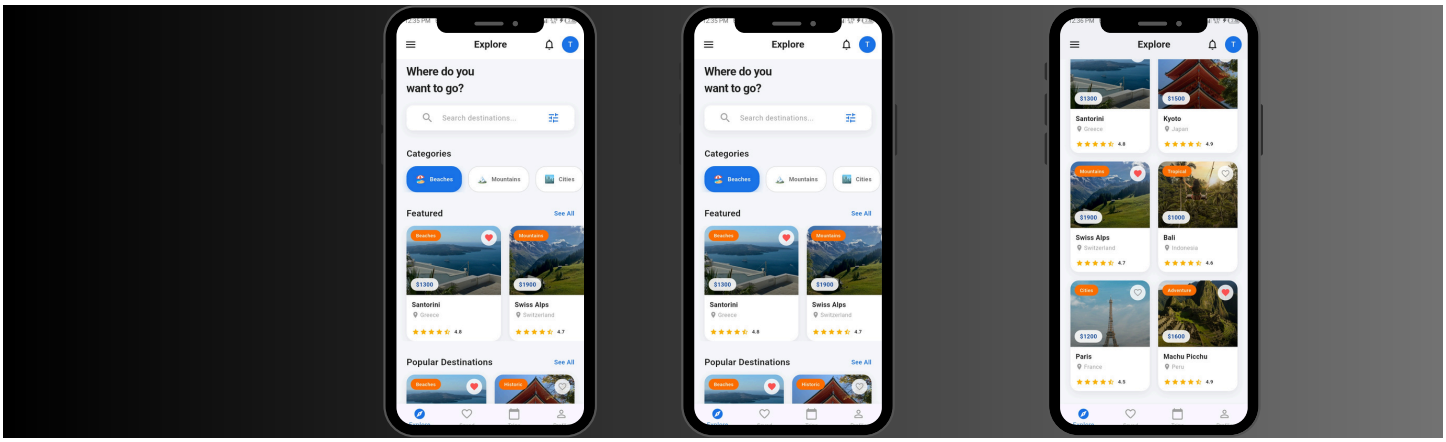
The Destination model is defined in lib/models/destination.dart as an immutable class with const constructor support.

4. Screen Descriptions and Screenshots

4.1 Home Screen (Complex Dashboard)

The Home Screen serves as the main dashboard of the app. It features an AppBar with a menu icon, notification icon, and user avatar. Below the AppBar is a greeting message and a search bar (UI only, non-functional). A horizontal scrolling list of category chips allows visual filtering. Featured destinations appear in a horizontal scrollable list, while all destinations are displayed in a 2-column grid using GridView.builder. Each destination card shows an image with gradient overlay, favorite icon, category chip, price tag, destination name, country, and star rating. Tapping any card navigates to the Detail Screen.

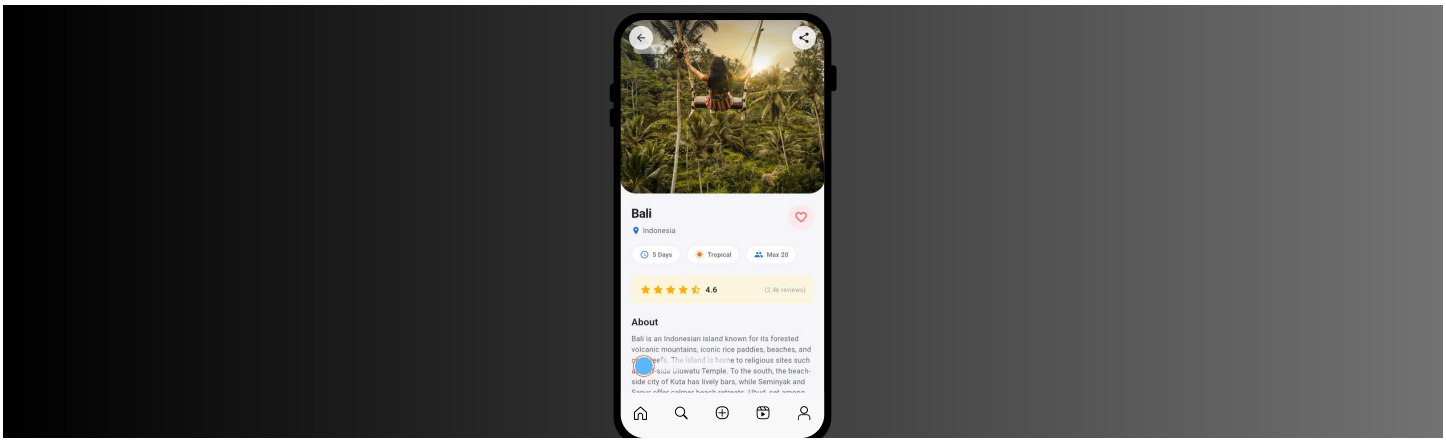
Home Screen



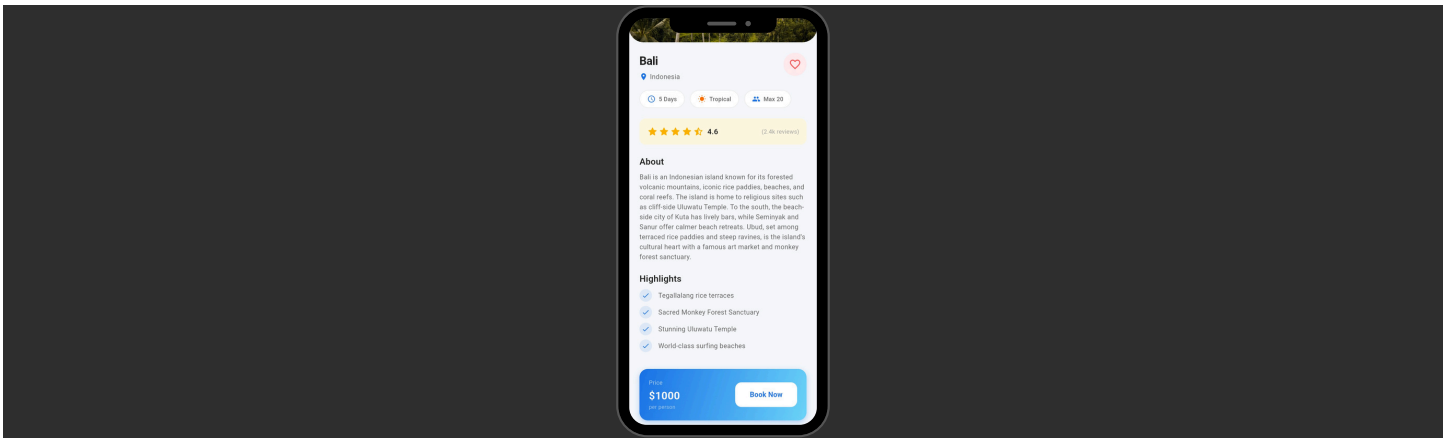
4.2 Detail Screen

The Detail Screen displays comprehensive information about a selected destination. It opens with a large header image (340px tall) with rounded bottom corners, a gradient overlay for readability, and circular back/share buttons positioned over the image using Stack and Positioned. Below the image, the destination name, country, and a favorite button are shown. Three InfoChip widgets display quick facts (duration, category, group size). A rating section with star icons and review count is enclosed in an amber-tinted container. The scrollable description text and a highlights list with checkmark icons follow. At the bottom, a gradient-styled booking bar shows the price and a prominent Book Now button that navigates to the Booking Screen.

Detail Screen (Top half)



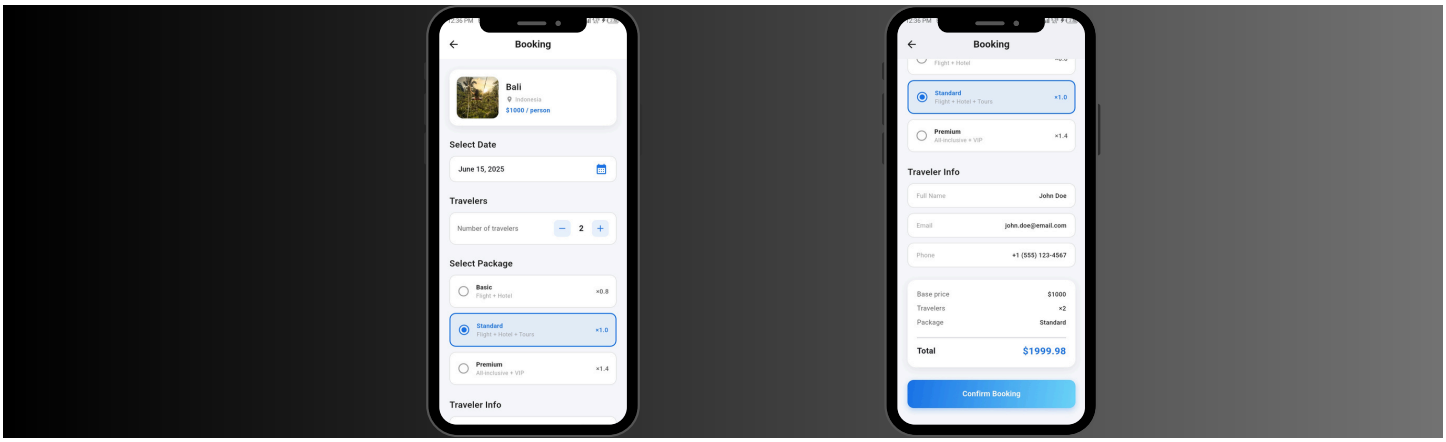
Detail Screen (Bottom half with Book Now)



4.3 Booking Screen

The Booking Screen provides a form-style booking experience. At the top, a summary card shows the destination image, name, country, and per-person price. The user can select a travel date from a dropdown, adjust the number of travelers using increment/decrement buttons, and choose between Basic, Standard, or Premium packages displayed as selectable cards with radio-style indicators. Hard-coded traveler information (name, email, phone) is displayed in read-only styled fields. A price summary section calculates the total dynamically based on selections. A gradient Confirm Booking button triggers both a success dialog and a snackbar notification.

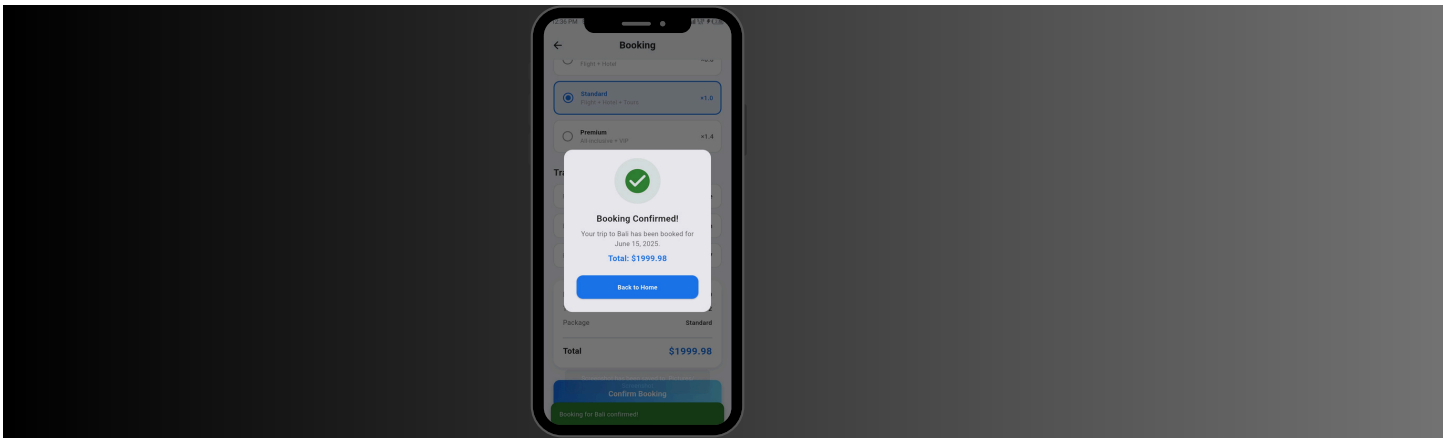
Booking Screen



4.4 Success Dialog and SnackBar

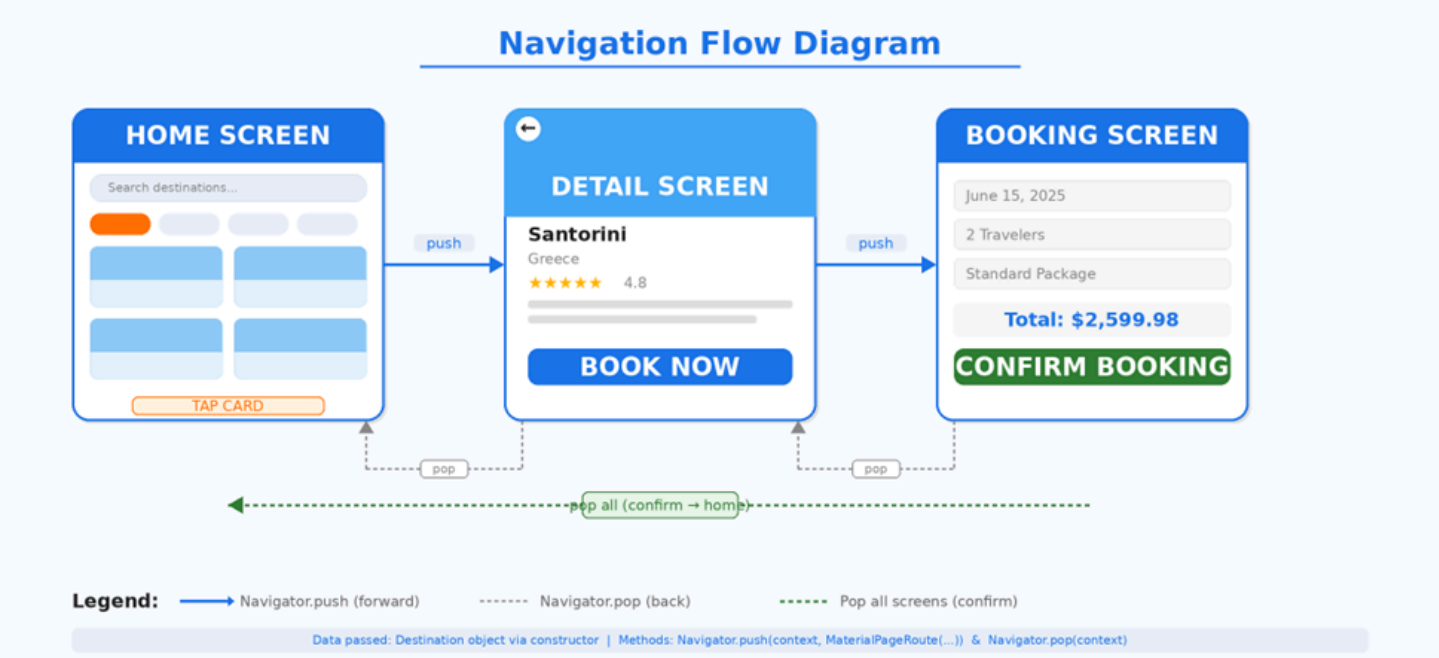
After pressing Confirm Booking, an AlertDialog appears with a green checkmark icon inside a circular container, a confirmation message showing the destination name and selected date, the total price in blue, and a Back to Home button. Simultaneously, a floating SnackBar with green background appears at the bottom confirming the booking. The Back to Home button pops all screens and returns to the Home Screen.

Success Dialog with SnackBar



5. Navigation Flow Diagram

The app implements navigation using Navigator.push and Navigator.pop. Hard-coded Destination objects are passed between screens via constructor parameters.



Forward navigation uses Navigator.push with MaterialPageRoute, passing the Destination object via constructor. Back navigation uses Navigator.pop. The confirmation dialog pops all three screens to return directly to the Home Screen.

6. Layout Choices and Explanations

6.1 Home Screen Layout

The Home Screen uses a SingleChildScrollView wrapping a Column to create a vertically scrollable dashboard. A horizontal ListView.builder renders category chips for browsing. Another horizontal ListView.builder shows featured destinations in a card carousel. The main content uses GridView.builder with a SliverGridDelegateWithFixedCrossAxisCount (2 columns) to display all destinations in a responsive grid layout. The childAspectRatio of 0.72 ensures cards have the right proportion of image to text content. A BottomNavigationBar provides tab navigation UI.

6.2 Detail Screen Layout

The Detail Screen uses a SingleChildScrollView to allow the entire page to scroll smoothly. The header image uses a Stack widget with Positioned children for the back button and share button overlay. A Container with gradient decoration creates the dark overlay on the image for text readability. The info chips use a Wrap widget to flow across multiple lines if needed. The rating section is enclosed in an amber-shaded Container with

rounded corners. The booking bar at the bottom uses a Container with LinearGradient decoration and contains a Row with price information and an ElevatedButton.

6.3 Booking Screen Layout

The Booking Screen uses a form-style layout with clearly separated sections. The destination summary uses a Row with an image and Column of text details inside a shadowed Container. The date picker uses DropdownButton inside a bordered Container. The traveler count uses a Row with custom increment/decrement buttons. Package selection uses List.generate to create AnimatedContainer cards that visually respond to selection state. The price summary is contained in a shadowed Card-style Container with a Divider separating line items from the total. The confirm button uses a DecoratedBox with gradient under a transparent ElevatedButton.

7. Widgets Used (27 Widgets)

The following table lists all 27 distinct Flutter widgets used in this project, exceeding the minimum requirement of 18:

#	Widget	Where Used
1	**MaterialApp**	main.dart — Root widget of the entire app
2	**Scaffold**	All 3 screens — Provides AppBar, body, bottom nav structure
3	**AppBar**	Home Screen, Booking Screen — Top navigation bar with icons
4	**Column**	All screens — Vertical layout of content sections
5	**Row**	Cards, price bars, counters — Horizontal layout of elements
6	**Container**	Everywhere — Styled boxes with decoration, shadows, gradients
7	**Stack**	Destination cards, detail header — Layered widget positioning
8	**Positioned**	Favorite icon, price chip, back/share buttons
9	**GridView.builder**	Home Screen — 2-column destination grid with lazy building
10	**ListView.builder**	Home Screen — Horizontal categories and featured lists
11	**SingleChildScrollView**	Detail Screen, Booking Screen — Scrollable content
12	**ClipRRect**	All image displays — Rounded corner clipping
13	**Image.asset**	Cards, detail header, booking summary — Asset images
14	**Icon**	Stars, location pins, navigation, info chips

15	**TextField**	Home Screen — Search bar (UI only)
16	**ElevatedButton**	Book Now, Confirm Booking, Back to Home
17	**GestureDetector**	Card taps, category chips, counter buttons
18	**AlertDialog**	Booking confirmation success dialog
19	**DropdownButton**	Booking Screen — Travel date selector
20	**BottomNavigationBar**	Home Screen — Tab navigation (Explore, Saved, Trips, Profile)
21	**CircleAvatar**	Home Screen AppBar — User profile icon
22	**Wrap**	Detail Screen — Info chips layout
23	**AnimatedContainer**	Category chips, package selection — Smooth transitions
24	**SafeArea**	Home Screen — Avoids system UI overlap
25	**Divider**	Booking Screen — Price summary separator
26	**SnackBar**	Booking confirmation floating notification
27	**DecoratedBox**	Confirm button — Gradient behind transparent button

8. Custom Reusable Widget Classes

Five custom reusable widget classes were created in the lib/widgets/ directory to promote code reuse and clean separation of UI components:

Widget Class	Description	Used In
RatingStars	Displays 1–5 star icons (full, half, empty) w	DestinationCard, DetailScreen
DestinationCard	Complete card with image, gradient overla	HomeScreen (grid and featured list)
SectionTitle	Section heading row with optional 'See All'	HomeScreen (Categories, Featured, Popular)
CategoryChip	Styled chip with emoji icon and label; supp	HomeScreen (horizontal category list)
InfoChip	Small badge with icon and text label for dis	DetailScreen (duration, category, group size)

9. UI Features

9.1 Gradients

Two gradients are defined in AppTheme and used throughout the app. The primaryGradient (blue to light blue, top-left to bottom-right) is applied to the Book Now price bar on the Detail Screen and the Confirm Booking button on the Booking Screen. The cardOverlayGradient (transparent to black, top to bottom) is applied over destination card images to ensure text readability on the overlaid content.

9.2 Shadows

Two shadow configurations are defined in AppTheme. The cardShadow (black at 8% opacity, 12px blur, 4px vertical offset) is applied to destination cards, the search bar, and summary containers. The buttonShadow (primary blue at 30% opacity, 8px blur, 4px offset) is applied to selected category chips and the gradient booking bars to create depth.

9.3 Consistent Color Theme

The AppTheme class defines a unified color palette used across all screens: primaryColor (#1A73E8 blue) for interactive elements and accents, accentColor (#FF6F00 orange) for category chips and highlights, three text colors (dark, medium, light) for content hierarchy, starColor (#FFB300 amber) for rating stars, and successColor (#2E7D32 green) for confirmation dialogs and snackbars. A full ThemeData object is configured in AppTheme and applied in MaterialApp.

9.4 Images from Assets

Six high-quality landscape images are stored in the assets/images/ directory and declared in pubspec.yaml. Each Destination object references its image via the imageAsset field. All images are loaded using Image.asset with BoxFit.cover for consistent display. An errorBuilder fallback is provided on every image to gracefully handle missing assets by showing a tinted placeholder with an icon.

9.5 Responsive Spacing

Consistent spacing is achieved through SizedBox widgets for vertical gaps, symmetric EdgeInsets for horizontal padding, and the AppTheme borderRadius constants (cardRadius: 16px, buttonRadius: 12px, chipRadius: 20px). The GridView uses childAspectRatio of 0.72 to maintain proper card proportions across different screen sizes.

10. Conclusion

The Travel Explorer app successfully demonstrates advanced Flutter UI design and navigation concepts. The project meets and exceeds all lab requirements: it implements three fully-designed screens with complex layouts, uses 27 distinct widgets (requirement: 18), creates 5 custom reusable widget classes, applies gradients and shadows throughout, loads images from assets, maintains a consistent color theme via the AppTheme class, and implements proper navigation with data passing between screens using Navigator.push and Navigator.pop.